OPERATING SYSTEMS ORANGE PROBLEM

NAME: KRISHNA JHA

SRN: PES1UG24CS236

TODO 1: Inter-Process Communication (IPC)

The Intuition: We need a way for multiple Card Spawner threads to send live text updates to a single Battle Logger thread. To do this, we use a unidirectional data channel. The Spawners will write to the "write end" of the channel, and the Logger will read from the "read end."

The Tool: `<unistd.h>`

- `int pipe(int pipefd[2]);`
  - This system call creates a pipe. It takes an array of two integers.
  - `pipefd[0]` becomes the file descriptor for the **read end**.
  - `pipefd[1]` becomes the file descriptor for the **write end**.
  - Returns `0` on success, or `-1` if it fails.

```
// TODO 1: IPC - Create the Battle Log Pipe
if (pipe(battle_log_pipe) == -1) {
perror("pipe");
exit(1);
```

TODO 2: Deadlock Prevention (Elixir Management)

The Intuition: `available_elixir` is a globally shared variable. If multiple threads try to subtract from it simultaneously, your server will experience a race condition and the math will corrupt. Furthermore, to avoid deadlocks, a Spawner must lock the Elixir pool, check if it has enough Elixir to cover the *entire* cost of the card, deduct it if true, and then unlock the pool.

The Tool: `<pthread.h>`

- `pthread_mutex_lock(pthread_mutex_t *mutex);`
  - Acquires a lock. If another thread already owns it, the calling thread will pause and wait until it is unlocked.
- `pthread_mutex_unlock(pthread_mutex_t *mutex);`
  - Releases the lock so other threads can access the critical section.

```
// TODO 2: SYNCHRONIZATION - Initialize Sync Primitives
sem_init(&arena_empty_slots, 0, arena_capacity);

pthread_mutex_init(&arena_mutex, NULL);
pthread_mutex_init(&elixir_lock, NULL);
```

TODO 3 & 4: Synchronization (The Bounded Buffer)

The Intuition: The Arena is an array with a strict limit (`arena_capacity`). You are implementing a classic **Producer-Consumer** pattern.

1. A Spawner (Producer) cannot deploy a troop if the Arena is full. It must wait for an "empty slot."
2. Deploying the troop into `arena[deploy_idx]` and updating `deploy_idx` is a *Critical Section*. Only one thread can do this at a time.
3. Once deployed, the Spawner must signal that a new "filled slot" exists in the Arena.

The Tools: `<semaphore.h>` **and** `<pthread.h>`

- `int sem_init(sem_t *sem, int pshared, unsigned int value);`
  - Initializes a semaphore. `pshared` should be `0` (shared between threads of the same process). `value` is the starting count. (Hint: Think about how many empty slots exist when the game starts vs. how many filled slots).
- `int sem_wait(sem_t *sem);`
  - Decrements the semaphore. If the value is `0` (e.g., no empty slots left), the thread goes to sleep until the value goes up.
- `int sem_post(sem_t *sem);`
  - Increments the semaphore, potentially waking up a sleeping thread.

```
// TODO 3: SIGNALS - Handle Match Over
// ------------------------------------------------------------

signal(SIGINT, match_over_handler);
```

```c
// ------------------------------------------------------------
// TODO 4: DEADLOCK PREVENTION - Spend Elixir Atomically
// ------------------------------------------------------------
bool spend_elixir(int cost) {
    bool deployed = false;

    pthread_mutex_lock(&elixir_lock);

    if (available_elixir >= cost) {
        available_elixir -= cost;
        deployed = true;
    }

pthread_mutex_unlock(&elixir_lock);

    return deployed;
}

void generate_elixir(int amount) {
    pthread_mutex_lock(&elixir_lock);
    available_elixir += amount;
    pthread_mutex_unlock(&elixir_lock);
}

// Card Spawner Thread (Producer)
void *card_spawner(void *id_ptr) {
    int id = *(int *)id_ptr;
    char log_msg[128];

    while (1) {
        int elixir_cost = (rand() % 4) + 1;

        if (spend_elixir(elixir_cost)) {
```

TODO 5: Signals (Handling the Rage Quit)

The Intuition: When a player hits `Ctrl+C` in the terminal, the Operating System generates a specific software interrupt called `SIGINT`. By default, this immediately kills the process. We want to override this default behavior so that when `SIGINT` is fired, the server routes the execution to our custom `match_over_handler` function, allowing us to print the final game stats before shutting down.

The Tool: `<signal.h>`

- `void (*signal(int signum, void (*handler)(int)))(int);`
  - *Don't let the syntax scare you!* You just call `signal(SIGNAL_MACRO, function_name);`
  - `signum`: The macro for the signal you want to catch (in this case, `SIGINT`).
  - `handler`: The name of the function you want to execute when the signal is caught.

```
        // ---------------------------------------------------------
        // TODO 5a: BOUNDED BUFFER (Producer) — Deploy troop into Arena
        sem_wait(&arena_empty_slots);

        pthread_mutex_lock(&arena_mutex);

        arena[deploy_idx] = id;
        deploy_idx = (deploy_idx + 1) % arena_capacity;

        pthread_mutex_unlock(&arena_mutex);
        sprintf(log_msg, "[%s] Spawner %d deployed a troop (Cost: %d Elixir).\n", STUDENT_SRN, id, elixir_cost);
        write(battle_log_pipe[1], log_msg, strlen(log_msg) + 1);

        total_troops_deployed++;

        // ---- Troop is alive on the Arena ----
        sleep(1);
        generate_elixir(elixir_cost);
        // ---- Troop has now died ----

        // ---------------------------------------------------------
        // TODO 5b: BOUNDED BUFFER — Free the Arena slot
        sem_post(&arena_empty_slots);
    } else {
        usleep(150000);
    }
}
```

OUTPUT:

```
[PES1UG24CS236] Clash Royale Match Starting...
[PES1UG24CS236] Arena Capacity: 6 | Max Elixir: 12
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 1 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 3 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 2 Elixir).
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 3 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 2 Elixir).
[PES1UG24CS236] Spawner 3 deployed a troop (Cost: 2 Elixir).
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 3 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 4 Elixir).
[PES1UG24CS236] Spawner 3 deployed a troop (Cost: 3 Elixir).
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 1 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 4 Elixir).
[PES1UG24CS236] Spawner 3 deployed a troop (Cost: 2 Elixir).
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 2 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 1 Elixir).
[PES1UG24CS236] Spawner 3 deployed a troop (Cost: 2 Elixir).
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 3 Elixir).
[PES1UG24CS236] Spawner 2 deployed a troop (Cost: 3 Elixir).
[PES1UG24CS236] Spawner 3 deployed a troop (Cost: 4 Elixir).
[PES1UG24CS236] Spawner 1 deployed a troop (Cost: 3 Elixir).
^C

=== KING TOWER DESTROYED! ===
[PES1UG24CS236] MATCH OVER (SIGINT Caught).
[PES1UG24CS236] Total Troops Deployed: 21
Closing server connections...
```